MODULE 04 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda II – Concurrency and Cold Starts

How Lambda scales, what throttling looks like, and the three ways to make a cold start stop hurting.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 4 of 34

admin@sthithapragna.com

Concurrency is not requests per second. It is **how many requests are in flight at the same instant**.

The formula worth memorising

- **Concurrency = requests per second x average duration in seconds**
- 100 requests/sec at 200 ms = 100 x 0.2 = **20 concurrent executions**.
- 100 requests/sec at 3 seconds = 100 x 3 = **300 concurrent executions**.
- Making a function faster reduces concurrency, which reduces throttling risk.
- A slow dependency raises concurrency without any change in traffic.

> ON THE EXAM
>
> Exam questions give you a request rate and a duration and expect you to produce a number. Practise the multiplication until it is automatic.

The account concurrency limit

- **1,000 concurrent executions** by default, per account, per Region.
- This is a **soft limit**. Raise it through Service Quotas or a support case.
- **Shared across every function in the account.** One noisy function can starve the rest.
- Provisioned concurrency is drawn from this same pool.
- At least **100** must remain unreserved for functions with no reservation of their own.

> ON THE EXAM
>
> Soft limit, so 'request a quota increase' is a legitimate answer here - unlike the 15-minute timeout, where it never is.

How fast Lambda can scale

- **1,000 new execution environments every 10 seconds**, per function, per Region.
- Equivalently, 10,000 additional requests per second every 10 seconds.
- **It is per function.** One function's spike no longer steals headroom from another.
- It does **not accrue**. Idling does not bank capacity for a later burst.
- Going from 0 to 30,000 concurrency takes roughly five minutes of sustained demand.

> ON THE EXAM
>
> Having concurrency available and being allowed to use it instantly are different things. Lambda rate-limits how fast environments appear.

Currency note: scaling was rebuilt

- **Old model:** an account-wide burst pool of 500, 1,000 or 3,000 depending on Region, refilling at 500 per minute.
- That pool was **shared**, so functions competed. A classic noisy-neighbour problem.
- **New model, since 6 December 2023:** 1,000 every 10 seconds, **per function**. Up to 12x faster.
- The noisy-neighbour effect on **scaling rate** is gone entirely.
- The account **concurrency limit** is still shared. Only the scaling rate became per-function.

> ON THE EXAM
>
> Plenty of question banks and courses still teach the old burst-pool model with its 500-per-minute refill. If you meet those numbers, they are out of date.

What happens when a request arrives

01

Warm?

An idle environment exists. Reuse it. No cold start.

02

Scale?

None free, but concurrency and scaling rate allow more. Build one. Cold start.

03

Throttle

At the limit, or scaling too fast. Reject with a 429.

ON THE EXAM

Only the middle path costs you a cold start, and only the third path produces an error. Most invocations take the first path.

What throttling actually does

INVOCATION TYPE	BEHAVIOUR WHEN THROTTLED
Synchronous, for example API Gateway	Immediate 429 TooManyRequestsException. The caller must retry
Asynchronous, for example S3 or SNS	Lambda retries internally for up to 6 hours with backoff
Event source mapping, SQS	The message stays on the queue and is retried
Event source mapping, Kinesis or DynamoDB	The shard blocks and retries until success or expiry

ON THE EXAM

Synchronous throttling is visible to the user immediately. Asynchronous throttling is usually invisible until you look at the metrics.

Maximum concurrency is not reserved concurrency

- **Reserved concurrency** is set on the **function**. It caps and reserves account capacity.
- **Maximum concurrency** is set on the **event source mapping**. It caps only that source.
- One function with several queues can give each queue its own maximum.
- On reaching the maximum, Lambda simply stops reading more messages. Nothing is thrown away.
- Reserved concurrency at the same value would throttle, sending messages back for retry.

> ON THE EXAM
>
> Throttling a queue-driven function wastes retries. Maximum concurrency slows the poller instead, which is the cleaner control for SQS.

Concurrency for event source mappings

- With SQS, Lambda **polls** rather than being pushed. The poller decides how fast to scale.
- Standard queues start at **5 concurrent batches** and add up to **300 instances per minute**.
- **FIFO queues scale to the number of active message groups**, and no further.
- So a FIFO queue with one message group gives you a concurrency of exactly one.
- Lambda deliberately slows scaling while invocations are failing, then resumes once they succeed.

> ON THE EXAM
>
> The FIFO point decides questions. If ordering matters and every message shares a group ID, no amount of concurrency configuration will parallelise it.

Reserved concurrency

- **Caps** the function. It can never exceed the reserved number.
- **Guarantees** the function. That capacity is carved out and nobody else can take it.
- Both jobs, one setting. That is what makes it confusing and examinable.
- It is subtracted from the account's unreserved pool the moment you set it.
- **No extra cost.** You are partitioning capacity, not buying it.

> ON THE EXAM
>
> Use it to protect a downstream database from too many connections, and to stop a batch function starving a customer-facing one.

Reserved concurrency of zero

- Setting reserved concurrency to **0** throttles every invocation of that function.
- The fastest way to stop a misbehaving function without deleting it.
- Useful during an incident, or to pause a consumer while you fix a downstream system.
- Configuration, event source mappings and code all stay intact. Reversible instantly.
- Asynchronous events still arrive and are retried, so they can pile up while it is off.

> ON THE EXAM
>
> When a question asks how to immediately stop a function without losing its configuration, this is the answer.

Provisioned concurrency

- Lambda keeps N environments **initialised and waiting**. INIT has already run.
- Configured on a **version or an alias**. Never on $LATEST.
- **You pay for it by the hour**, whether or not any request arrives.
- Traffic beyond the provisioned amount **spills over** to on-demand, with cold starts.
- Watch **ProvisionedConcurrencySpilloverInvocations** to see if you under-provisioned.

> ON THE EXAM
>
> The only option that eliminates cold starts outright. It is also the only one with a standing bill.

Reserved or provisioned concurrency?

RESERVED CONCURRENCY

Controls how many

- Caps and guarantees a maximum
- Protects downstream systems
- Stops noisy functions starving others
- Free - it partitions existing capacity
- Does nothing about cold starts

PROVISIONED CONCURRENCY

Controls how fast

- Pre-initialises environments
- Removes cold start latency
- Set on a version or alias only
- Billed hourly, used or not
- Does nothing about capacity limits

THE TIEBREAKER

Reserved is about quantity. Provisioned is about latency. The names sound alike and they solve unrelated problems - this pair appears constantly.

Scaling provisioned concurrency automatically

- **Scheduled scaling** raises and lowers it at known times. Ideal for predictable daily peaks.
- **Target tracking** adjusts against a utilisation target as traffic moves.
- Together they avoid paying peak rates around the clock.
- Utilisation is ProvisionedConcurrentExecutions divided by allocated.
- Scaling up is not instant. Provisioning environments takes time, so schedule ahead of the spike.

> ON THE EXAM
>
> A scenario with a known daily or weekly traffic pattern and a cost constraint is usually pointing at scheduled scaling.

What actually causes a cold start

- **Scaling up.** More concurrent requests than warm environments.
- **Idle reaping.** Lambda reclaims environments after a period with no traffic.
- **Any deployment.** Publishing new code discards every warm environment.
- Large deployment packages and heavy imports make each one longer.
- Java and .NET suffer most. Python and Node.js are far quicker to start.

> ON THE EXAM
>
> Cold starts are a small fraction of invocations in steady traffic. They matter most for spiky traffic and latency-sensitive APIs.

Lambda SnapStart

- Lambda runs INIT once, snapshots the initialised environment, and **restores from the snapshot** on later cold starts.
- Cold starts for heavy Java frameworks can fall from seconds to a few hundred milliseconds.
- **No additional charge** for the feature itself.
- Runtimes: **Java 11+, Python 3.12+, .NET 8+**. Not Node.js, not Ruby.
- Works only on **published versions and aliases**. Never on $LATEST.

> ON THE EXAM
>
> The middle ground: most of the latency benefit of provisioned concurrency, without the standing bill. But only on supported runtimes.

What SnapStart will not do

CONSTRAINT	DETAIL
Provisioned concurrency	Mutually exclusive. You cannot use both on the same version
Amazon EFS	Not supported
Ephemeral storage	Cannot exceed the default 512 MB
$LATEST	Not supported. Published versions and aliases only
Node.js and Ruby	Not supported runtimes

ON THE EXAM

The first row is the highest-value fact here. A scenario that needs both is a scenario where one of them has to go.

The SnapStart uniqueness trap

- One snapshot becomes the starting state for **many** environments.
- So anything unique generated during INIT is **duplicated** across all of them.
- Random seeds, UUIDs, cached credentials and timestamps are all at risk.
- **Fix one:** generate unique values inside the handler, not at module load.
- **Fix two:** use a runtime hook that runs after restore, or a CSPRNG like SecureRandom.

> ON THE EXAM
>
> Symptoms are intermittent and nasty: duplicate IDs, identical 'random' values, stale credentials. AWS ships an open-source scanning tool for Java to catch it.

SnapStart or provisioned concurrency?

CHOOSE SNAPSTART WHEN

Latency matters, cost matters more

- Runtime is Java, Python or .NET
- No standing charge is acceptable
- Sub-second is good enough
- Traffic is spiky or unpredictable
- No EFS and /tmp stays at 512 MB

CHOOSE PROVISIONED CONCURRENCY WHEN

Latency is non-negotiable

- Any runtime, including Node.js
- Double-digit milliseconds required
- You will pay hourly for it
- Traffic is predictable enough to size
- You need EFS or larger /tmp

THE TIEBREAKER

SnapStart is free but restricted. Provisioned concurrency works everywhere and costs money. If the scenario stresses cost, lean SnapStart; if it stresses a hard latency SLA, lean provisioned.

The concurrency metrics that matter

METRIC	WHAT IT TELLS YOU
ConcurrentExecutions	How much concurrency is in use right now
UnreservedConcurrentExecutions	What is left in the shared pool
Throttles	Requests rejected. Any sustained value is a problem
ProvisionedConcurrentExecutions	How much of your provisioned capacity is busy
ProvisionedConcurrencySpilloverInvocations	Requests that overflowed to on-demand
ClaimedAccountConcurrency	Total claimed against the account quota

ON THE EXAM

Throttles above zero plus ConcurrentExecutions flat at a round number means you have hit a limit, not a bug.

Four scenarios. Commit to an answer **before** the walkthrough, including which options you eliminated and why.

> An account runs two Lambda functions. A customer-facing checkout function sits behind API Gateway and must never fail. A nightly report function is triggered by Amazon S3 and processes thousands of objects, scaling to hundreds of concurrent executions.
>
> During the nightly run, checkout requests start returning 429 errors. The account concurrency limit is the default 1,000. The business wants checkout protected, and is content for the report job to take longer if necessary.

What should the developer configure?

A. Request an account concurrency limit increase from 1,000 to 5,000

B. Configure provisioned concurrency on the checkout function

C. Configure reserved concurrency on the report function, capping it well below 1,000

D. Move the report function to a separate AWS Region

◆ PAUSE AND DECIDE

What should the developer configure?

A Raise the account limit

Buys headroom but fixes nothing structurally. The report function can still consume whatever it likes and starve checkout at a higher number.

B Provisioned concurrency on checkout

Solves cold starts, not capacity. Provisioned capacity still draws from the same account pool, and 429s are a concurrency problem.

C Reserved concurrency on the report function

Caps the noisy function so it can never consume the whole pool, and the business has already accepted a slower report.

D Separate Region

Concurrency limits are per Region so this would technically work, but it is a large architectural change to solve a configuration problem.

THE SIGNAL

Throttling is a capacity problem, so the answer is a capacity control. The sentence granting permission for a slower report is what makes capping acceptable.

A Java Spring Boot application runs as a Lambda function behind API Gateway. Cold starts take between 4 and 6 seconds and users notice. Traffic is unpredictable, arriving in bursts several times a day with long quiet periods between them.

The team's p99 latency target is under one second. Finance has rejected a proposal that adds a fixed monthly cost. The function writes small temporary files and does not use Amazon EFS.

What is the most appropriate way to reduce the cold start latency?

A Configure provisioned concurrency sized for peak traffic

B Increase the function's memory to 10,240 MB

C Create a scheduled EventBridge rule that invokes the function every 5 minutes to keep it warm

D Enable Lambda SnapStart on a published version and point the alias at it

◆ PAUSE AND DECIDE

What is the most appropriate way to reduce the cold start latency?

A Provisioned concurrency at peak

It would meet the latency target, but it bills hourly whether traffic arrives or not - exactly the fixed cost finance rejected, and wasteful given long quiet periods.

B Maximum memory

More CPU shortens INIT somewhat, but it will not turn a 5-second JVM start into under a second, and it raises the cost of every invocation.

C EventBridge warming

The classic hack. It keeps roughly one environment warm, so it does nothing for a burst that needs fifty, and it bills for invocations that do no work.

D SnapStart on a published version

Purpose-built for exactly this: heavy JVM initialisation, no additional charge, and every restored environment skips INIT rather than only the first.

THE SIGNAL

Three constraints line up: Java runtime, no fixed cost, unpredictable bursts. SnapStart is the only option that satisfies all three.

A team enables SnapStart on a Java function that generates a correlation ID for each execution environment at startup and uses java.util.Random, seeded once during initialisation, to produce short reference codes for customer orders.

After deployment, support reports that different customers are occasionally receiving the same order reference code. The problem is intermittent and never reproduces in a single-user test.

What is the cause, and what is the correct fix?

A SnapStart is corrupting memory. Disable SnapStart and use provisioned concurrency instead

B The seed and correlation ID are captured in the snapshot and shared by every restored environment. Generate them in the handler, or refresh them in an afterRestore runtime hook

C The function needs more memory so each environment gets its own random state

D Add a DynamoDB conditional write to detect duplicate codes and retry

◆ PAUSE AND DECIDE

What is the cause, and what is the correct fix?

A Disable SnapStart

Misdiagnoses it as a bug in Lambda. It is documented behaviour, and abandoning SnapStart gives up the benefit rather than fixing the code.

B Snapshot captured the seed; move generation after init

The correct diagnosis and the documented fix. One snapshot seeds many environments, so uniqueness must be created after restore.

C More memory

Memory has no bearing on random state. Every restored environment gets an identical copy regardless of size.

D Detect duplicates in DynamoDB

Papers over the defect. The generator is still producing colliding sequences, so collisions grow with concurrency and every order now costs an extra write.

THE SIGNAL

'Intermittent, and never in single-user testing' is the tell. It only appears once several environments restore from the same snapshot.

> A payments API on Node.js 22 must respond within 100 ms at p99, including cold starts. Traffic follows a strict and well-understood pattern: near zero overnight, rising sharply at 08:00, steady all day, falling after 18:00.
>
> The team has budget for the latency guarantee but does not want to pay peak rates for the sixteen quiet hours each day.

Which approach best meets both requirements?

A. Provisioned concurrency on an alias, with Application Auto Scaling scheduled scaling raising it before 08:00 and lowering it after 18:00

B. Enable SnapStart on a published version and point the alias at it

C. Reserved concurrency set to the expected daytime peak

D. Provisioned concurrency on $LATEST, fixed at the daytime peak all day

◆ PAUSE AND DECIDE

Which approach best meets both requirements?

A Provisioned concurrency plus scheduled scaling

Meets a 100 ms target that only pre-warmed environments can hit, and the schedule removes the cost during the sixteen quiet hours.

B SnapStart

Node.js is not a supported SnapStart runtime. Even if it were, restore latency does not reliably reach double-digit milliseconds.

C Reserved concurrency

Reserves capacity but pre-warms nothing. Every scale-up still cold starts, so the latency target is missed.

D Provisioned concurrency on $LATEST

Not possible - provisioned concurrency requires a version or alias. It also pays peak rates overnight, which the scenario rules out.

THE SIGNAL

Compare with scenario 2. Same problem, opposite answer, because the runtime and the cost constraint both flipped.

Currency note: Lambda Managed Instances

- Runs Lambda functions on **dedicated EC2-backed instances** in your account, keeping the Lambda programming model.
- Multiple concurrent invocations share an instance, rather than one environment per request.
- Aimed at **steady, high-volume** workloads where per-invocation pricing stops making sense.
- Compatible with EC2 Savings Plans, and gives access to newer processors.
- **Not in exam guide v2.1.** Answer the exam from the classic model: one environment, one concurrent request.

> ON THE EXAM
>
> Worth knowing because it changes the standard 'Lambda gets expensive at constant high load, so move to Fargate' answer. On the exam, that answer still stands.

What to carry into Lambda III

- Concurrency = requests per second x duration. Slower functions cost more concurrency.
- **Reserved is quantity. Provisioned is latency.** Free versus billed hourly.
- SnapStart: free, Java/Python/.NET, published versions only, and never with provisioned concurrency.
- Scaling is **1,000 per 10 seconds, per function**. The old shared burst pool is gone.
- Throttling means different things for sync, async and event source mappings.

> ON THE EXAM
>
> Next: event sources and failure handling - where that last point gets its own module, along with retries, DLQs and idempotency.